

EMI JOIN

Important: SQL has no SEMI JOIN keyword. A semi join is a logical operation implemented using EXISTS or IN.

What is a SEMI JOIN?

A SEMI JOIN returns:

-  Rows from the left table only
-  If at least one matching row exists in the right table

Unlike an INNER JOIN, it never returns columns from the right table.

Example

Customers

customer_id	name
1	Arpit
2	Aadz
3	Riya

Orders

order_id	customer_id
101	1
102	1
103	2

Need: Customers who have placed at least one order.

INNER JOIN

```
SELECT *  
FROM Customers c  
JOIN Orders o  
ON c.customer_id=o.customer_id;
```

Output

customer	order
Arpit	101
Arpit	102
Aadz	103

Duplicates.

SEMI JOIN

```
SELECT *
FROM Customers c
WHERE EXISTS
(
    SELECT 1
    FROM Orders o
    WHERE o.customer_id=c.customer_id
);
```

Output

customer
Arpit
Aadz

Only left table.

No duplicates.

EXISTS

```
SELECT *
FROM Customers c
WHERE EXISTS
(
    SELECT 1
    FROM Orders o
    WHERE o.customer_id=c.customer_id
);
```

Returns TRUE immediately after finding the first match.

IN

```
SELECT *
FROM Customers
WHERE customer_id IN
(
    SELECT customer_id
    FROM Orders
);
```

Also behaves like a SEMI JOIN.

INNER JOIN vs SEMI JOIN

INNER JOIN	SEMI JOIN
Returns columns from both tables	Returns only left table
Duplicates possible	No duplicates caused by matches
Needs full join	Only existence matters

LeetCode Questions

182. Duplicate Emails

```
SELECT Email
FROM Person p1
WHERE EXISTS
(
    SELECT 1
    FROM Person p2
    WHERE p1.Email=p2.Email
        AND p1.id!=p2.id
)
GROUP BY Email;
```

ANTI JOIN

SQL has no **ANTI JOIN** keyword.

Implemented using:

- NOT EXISTS ✓
- LEFT JOIN ... IS NULL ✓
- NOT IN ⚠

Definition

Return rows from left table that have no match in right table.

Example

Customers

id	name
1	Arpit
2	Aadz
3	Riya

Orders

customer_id
1
2

Need: Customers without orders.

Answer: Riya

Method 1 — NOT EXISTS (Recommended)

```
SELECT *
FROM Customers c
WHERE NOT EXISTS
(
    SELECT 1
    FROM Orders o
    WHERE o.customer_id=c.customer_id
);
```

Best solution.

Method 2 — LEFT JOIN

```
SELECT c.*
FROM Customers c
LEFT JOIN Orders o
ON c.customer_id=o.customer_id
WHERE o.customer_id IS NULL;
```

Method 3 — NOT IN

```
SELECT *
FROM Customers
WHERE customer_id NOT IN
(
    SELECT customer_id
    FROM Orders
);
```

NULL Pitfall

Orders

customer_id
1
NULL

Now **NOT IN** returns Nothing.

Because

1 NOT IN (1, NULL) ↓ **UNKNOWN**

Huge interview favorite.

Best Practice

Always prefer **NOT EXISTS**

1581. Customer Who Visited but Did Not Make Any Transactions

```
SELECT
    v.customer_id,
    COUNT(*) AS count_no_trans
FROM Visits v
LEFT JOIN Transactions t
ON v.visit_id=t.visit_id
WHERE t.transaction_id IS NULL
GROUP BY v.customer_id;
```

Multi-table Joins

Now instead of **A ↔ B** we have **A ↔ B ↔ C**

Example

Orders
↓
Customers
↓
Products

```
SELECT
    c.name,
    p.product_name,
    oi.quantity
FROM Customers c
JOIN Orders o
ON c.customer_id=o.customer_id
JOIN OrderItems oi
ON o.order_id=oi.order_id
JOIN Products p
ON oi.product_id=p.product_id;
```

Composite Key Joins

Sometimes One column is not enough.
Need multiple columns.

Example

Sales

|product_id|year|

Price

|product_id|year|

Need: Product price of same year.

Wrong

ON product_id

Could match wrong year.

Correct

```
SELECT *  
FROM Sales s  
JOIN Price p  
ON s.product_id=p.product_id  
AND s.year=p.year;
```

Composite Key (product_id, year)

1070. Product Sales Analysis III

Composite Join

```
SELECT
    s.product_id,
    s.year AS first_year,
    s.quantity,
    s.price
FROM Sales s
JOIN
(
    SELECT
        product_id,
        MIN(year) AS year
    FROM Sales
    GROUP BY product_id
) t
ON s.product_id=t.product_id
AND s.year=t.year;
```

Customers Who Bought All Products

Solution

```
SELECT
    customer_id
FROM Customer
GROUP BY customer_id
HAVING COUNT(DISTINCT product_key) =
(
    SELECT COUNT(*)
    FROM Product
);
```

List the Products Ordered in a Period

Solution

```
SELECT
    p.product_name,
    SUM(o.unit) AS unit
FROM Products p
JOIN Orders o
ON p.product_id = o.product_id
WHERE o.order_date BETWEEN '2020-02-01' AND '2020-02-29'
GROUP BY p.product_id, p.product_name
HAVING SUM(o.unit) >= 100;
```

LeetCode 1075 — Project Employees I

Solution

```
SELECT
    p.project_id,
    ROUND(AVG(e.experience_years), 2) AS average_years
FROM Project p
JOIN Employee e
ON p.employee_id = e.employee_id
GROUP BY p.project_id;
```

LeetCode 550 — Game Play Analysis IV

Solution

```
SELECT
    ROUND(
        COUNT(DISTINCT a2.player_id) /
        (SELECT COUNT(DISTINCT player_id) FROM Activity),
        2
    ) AS fraction
FROM
    (
        SELECT
            player_id,
            MIN(event_date) AS first_login
        FROM Activity
        GROUP BY player_id
    ) a1
JOIN Activity a2
ON a1.player_id = a2.player_id
AND a2.event_date = DATE_ADD(a1.first_login, INTERVAL 1 DAY);
```

LeetCode 1193 — Monthly Transactions I

Solution

```
SELECT
    DATE_FORMAT(trans_date, '%Y-%m') AS month,
    country,
    COUNT(*) AS trans_count,
    SUM(state = 'approved') AS approved_count,
    SUM(amount) AS trans_total_amount,
    SUM(
        CASE
            WHEN state = 'approved' THEN amount
            ELSE 0
        END
    ) AS approved_total_amount
FROM Transactions
GROUP BY
    DATE_FORMAT(trans_date, '%Y-%m'),
    country;
```